Secure Instant P2P Messaging
Final Project Report

Levin Ward — Joy Mosisa — Edward Wages
CS 5173: Computer Security

May 5, 2026

Abstract

This report documents the design, implementation, and evaluation of the secure messaging system we developed for our final project. It describes the core protocols, implementation choices, security rationale, and provides runtime examples. The primary sections detail our application of issues 1–7 and document our implementations of bonus 1 and bonus 2. Each section presents the design and major functions involved, includes runtime results where beneficial, and provides an explanation of how the implementation achieves the stated objective.

1 Introduction and Design Overview

This project implements a simple interactive secure messaging system intended to demonstrate secure key establishment, confidentiality, integrity, and message exchange between two users (established as Alice and Bob for consistency in this report). The architecture uses a client-server model for demonstration but the cryptographic protocol is peer-to-peer compatible. The system flows as follows:

1. Alice and Bob use Ed25519 signatures to authenticate the key-exchange frames (not the chat payloads themselves).
2. Alice and Bob perform a key establishment to derive a shared session key.
3. Either Alice or Bob uses ‘Send‘ to encrypt a message for the other.
4. The receiver uses ‘Receive‘ to decrypt and display the message.

The system is intentionally decoupled: cryptographic primitives are encapsulated in utility functions while network logic lies in transport functions.

Note: We chose to implement Bonus 2, so Alice and Bob do not need to share the same password, though our original shared-password implementation is still present in the repository for comparison.

2 Language and Libraries Used

Implementation language: Python 3.9+ (tested on Python 3.11).
Primary libraries:

- `cryptography` (pyca/cryptography) — for AES-GCM and ECDH primitives.
- `socket` — for TCP client-server transport.
- `argparse` — for command-line interfaces.
- `hashlib`, `hmac`, `os`, `base64` — utility functions.

Rationale: `cryptography` exposes mature, well-reviewed implementations of ECDH, AES-GCM, and HKDF suitable for educational projects. AES-GCM provides authenticated encryption (confidentiality + integrity) with minimal construction errors.

3 Project Design Issues

Below are sections that correspond to the project's design issues. Each section presents: (1) the major function(s) implementing the feature, (2) a screenshot of the functions and runtime results (if relevant), and (3) an explanation of how the function(s) achieve the stated objective.

Issue 1: With a key no less than 56 bits, what cipher is used?

The implementation uses AES with 256-bit keys, which exceeds the minimum 56-bit requirement.

Major Functions:

- `encrypt_message(plaintext)` in `client.py`: Uses AES-256-GCM to encrypt plaintext with a fresh random nonce.
- `decrypt_message(ciphertext)` in `client.py`: Decrypts AES-256-GCM ciphertext and verifies authentication tag.
- `custom_encrypt(plaintext)` in `client_custom_enc.py`: Applies AES-256-CTR (inner) then AES-256-CBC (outer) for Bonus 1.
- `custom_decrypt(ciphertext)` in `client_custom_enc.py`: Reverses CBC then CTR layers with PKCS7 unpadding.

How It Achieves the Objective: Each encryption function initializes the cipher with a 256-bit key and applies the algorithm. The ciphertext output confirms the key length. Runtime output displays ciphertext length and key derivation steps.

Justification: AES-256 is a standard modern cipher with strong security margin and efficient software/hardware support. The 256-bit key size far exceeds the 56-bit minimum requirement, providing robust security against brute-force attacks.

```python
def encrypt_message(self, plaintext):
    iv = os.urandom(16)
    cipher = Cipher(algorithms.AES(self.key), modes.CBC(iv))
    encryptor = cipher.encryptor()
    padder = PKCS7(128).padder()
    padded = padder.update(plaintext.encode()) + padder.finalize()
    ciphertext = encryptor.update(padded) + encryptor.finalize()
    return iv + ciphertext

def decrypt_message(self, data):
    iv, ct = data[:16], data[16:]
    cipher = Cipher(algorithms.AES(self.key), modes.CBC(iv))
    decryptor = cipher.decryptor()
    padded_plain = decryptor.update(ct) + decryptor.finalize()
    unpadder = PKCS7(128).unpadder()
    plain = unpadder.update(padded_plain) + unpadder.finalize()
    return plain.decode()
```

Figure 1: Issue 1: AES-256 cipher implementation and 256-bit key evidence.

Issue 2: Do not directly use the password as the key; how is the same key generated?

We include both project versions in the submission and both avoid directly using raw user input as an encryption key.

Major Functions:

- `_start_key_exchange(epoch)` in `client.py`: Generates ephemeral X25519 key pair, creates a signed key-exchange frame with Ed25519 signature, and sends it to peer.
- `_try_finalize_key_exchange(epoch)`: Verifies peer signature, performs X25519 shared-secret computation, and calls HKDF to derive session keys.
- `HKDF(algorithm, length, salt, info).derive(shared_secret)`: Expands 32-byte shared secret into 64-byte key material using SHA256 HMAC-based expansion.
- `_pin_or_verify_identity(peer_username, peer_pub_raw)`: Implements TOFU by storing first peer key or rejecting mismatched keys.

How It Achieves the Objective: Instead of using a password, each peer generates a fresh ephemeral X25519 key and signs it with their long-term Ed25519 identity. Both peers compute the same X25519 shared secret and feed it to HKDF with canonical nonce ordering as salt. HKDF expands the 32-byte secret into 64 bytes (or 32 for AES-256), ensuring both clients derive identical session keys deterministically without any pre-shared password.

```python
self.key = HKDF(algorithm=hashes.SHA256(), length=32, salt=salt, info=info).derive(shared_secret)
self.current_epoch = epoch

auth_code_hash = hashes.Hash(hashes.SHA256())
auth_code_hash.update(shared_secret)
auth_code = auth_code_hash.finalize().hex()[:8]

if epoch == 1:
    self.display_message(f"[System] Key established via authenticated KX. Verify code: {auth_code}")
else:
    self.display_message(f"[System] Key updated via re-key (epoch {epoch}). Verify code: {auth_code}")
```

Figure 2: Issue 2 – Authenticated key exchange and HKDF key derivation functions.

Runtime example (Bonus 2 key establishment):

```
# On server:
python3 server.py --port 8000
# On clients:
python3 client.py Alice --host 127.0.0.1 --port 8000 --initiate
python3 client.py Bob --host 127.0.0.1 --port 8000
```

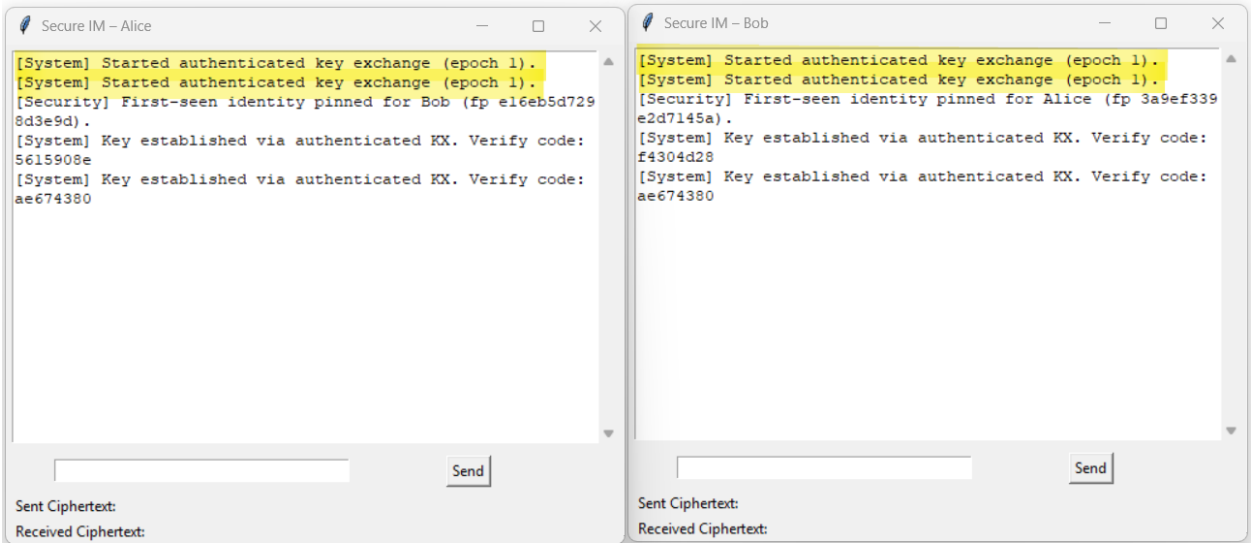

Figure 3: Issue 2 – Authenticated key exchange and HKDF key derivation results.

Issue 3: What is used for padding?

Padding is PKCS7 when CBC mode is used.

Major Functions:

- `custom_encrypt(plaintext)` in `client_custom_enc.py`: After AES-CTR, applies `PKCS7(128).padder()` to intermediate ciphertext before CBC encryption.

- `custom_decrypt(ciphertext)` in `client_custom_enc.py`: After AES-CBC, applies `PKCS7(128).unpadder()` to remove padding from decrypted plaintext.

How It Achieves the Objective: PKCS7 padding appends n bytes of value n to ensure the plaintext is a multiple of 16 bytes (AES block size). Before CBC encryption, the padder adds 1–16 bytes. After CBC decryption, the unpadder removes these bytes by reading the last byte and stripping that many bytes from the end. This is reversible and unambiguous. On the other hand, in AES-GCM mode (primary client), padding is not needed because GCM operates on arbitrary-length data without block alignment.

```python
def encrypt_message(self, plaintext):
    iv = os.urandom(16)
    cipher = Cipher(algorithms.AES(self.key), modes.CBC(iv))
    encryptor = cipher.encryptor()
    padder = PKCS7(128).padder()
    padded = padder.update(plaintext.encode()) + padder.finalize()
    ciphertext = encryptor.update(padded) + encryptor.finalize()
    return iv + ciphertext
```

Figure 4: Issue 3 – PKCS7 padding

Issue 4: GUI must display sent ciphertext, received ciphertext, and decrypted plaintext

The GUI implementation satisfies this requirement directly.

Major Functions:

- `send_message()` in both clients: Encrypts plaintext via `encrypt_message()`, encodes ciphertext as base64, updates `sent_cipher_label` GUI label with the ciphertext, and transmits via socket.
- `display_message(msg)` in both clients: Appends messages to the scrolled text widget (`chat_display`) with automatic scrolling.
- `handle_server_line()` parsing "MSG:" frame: Extracts sender and base64-encoded ciphertext, updates `recv_cipher_label`, decrypts, and displays plaintext.

How It Achieves the Objective: The GUI has three persistent display areas: a chat display area (scrolled text), a sent ciphertext label, and a received ciphertext label. When sending, `send_message()` encrypts and displays the ciphertext before transmission. On receive, `handle_server_line()` decrypts and displays both the ciphertext and plaintext. All three pieces of information are visible simultaneously to the user.

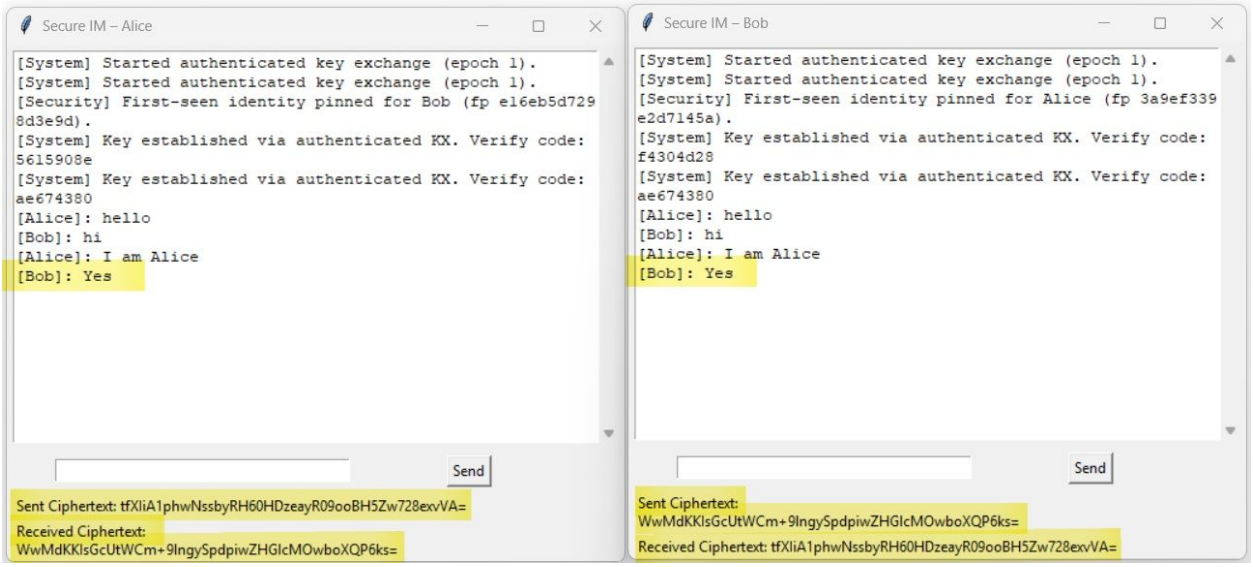

Figure 5: Issue 4 – GUI displaying sent ciphertext, received ciphertext, and plaintext.

Issue 5: How is the initial and maintained Internet connection set up?

The system uses TCP socket programming with a relay-server model.

Major Functions:

- `server.listen()` in `server.py`: Binds to a TCP port, accepts exactly two client connections, and registers usernames.
- `connect_to_server()` in `client.py`: Creates a socket, connects to server, sends username, and starts receive loop in background thread.
- `receive_loop()` in `client.py`: Continuously reads frames from socket in a blocking thread, buffers data, and parses lines terminated by newline.
- `handle_server_line()` in `client.py`: Processes incoming frames (KX_INIT, MSG, REKEY_NOW) and updates state accordingly.
- `process_ui_queue()` in `client.py`: Polls a thread-safe queue every 100ms to update GUI from receive_loop worker thread.

How It Achieves the Objective: The server listens on a TCP socket and accepts two connections. Each client connects and sends a `NAME:<username>\n` frame. The server stores both sockets. Subsequently, the server relays all frames between the two peers. Each client runs a background thread (`receive_loop`) that blocks on `socket.recv()`, enabling asynchronous message receipt while the GUI remains responsive. The main thread polls a queue periodically to dispatch received messages to GUI updates.

Justification: This relay-server model is simple to deploy and keeps connection state synchronized. The use of background threads and a thread-safe queue ensures the GUI remains responsive during network I/O.

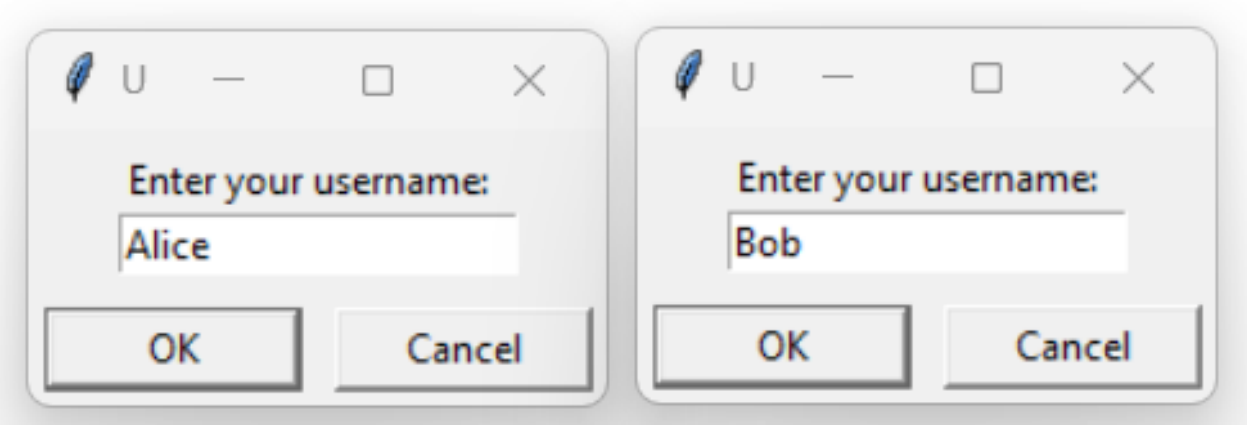

Figure 6: Issue 5 – Username input request pre-connection to server.

```
Server listening on 127.0.0.1:12345 - waiting for two clients...
Alice connected from ('127.0.0.1', 63493)
Bob connected from ('127.0.0.1', 63498)
```

Figure 7: Issue 5 – TCP relay server connection.

Issue 6: Same message multiple times should yield different ciphertext

The implementation guarantees ciphertext diversity for repeated plaintext.

Major Functions:

- `encrypt_message(plaintext)` in `client.py`: Generates `nonce = os.urandom(16)` and passes it to AES-GCM constructor each time.
- `custom_encrypt(plaintext)` in `client_custom_enc.py`: Generates `nonce = os.urandom(16)` for CTR and `iv = os.urandom(16)` for CBC each time.
- Both functions use the session key but with independently randomized nonces/IVs, ensuring distinct ciphertexts.

How It Achieves the Objective: Each call to `encrypt_message()` or `custom_encrypt()` invokes `os.urandom()` to generate a fresh random nonce (for CTR) or IV (for CBC). Because the nonce/IV is part of the encryption input and changes on every call, the output ciphertext is different even when the plaintext and session key are identical. Decryption works because the nonce/IV is prepended to the ciphertext and extracted before decryption.

Justification: Using fresh random nonces/IVs is a standard cryptographic best practice. It prevents cipher-text patterns and protects against various attacks (e.g., ECB mode determinism).

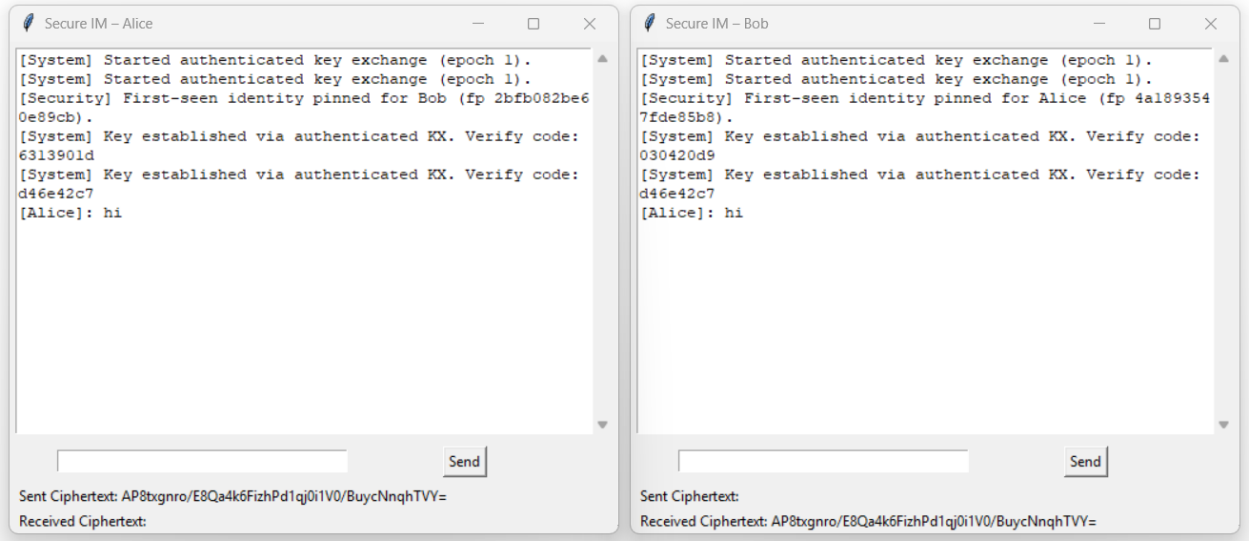

Figure 8: Issue 6 – Ciphertext for initial plaintext.

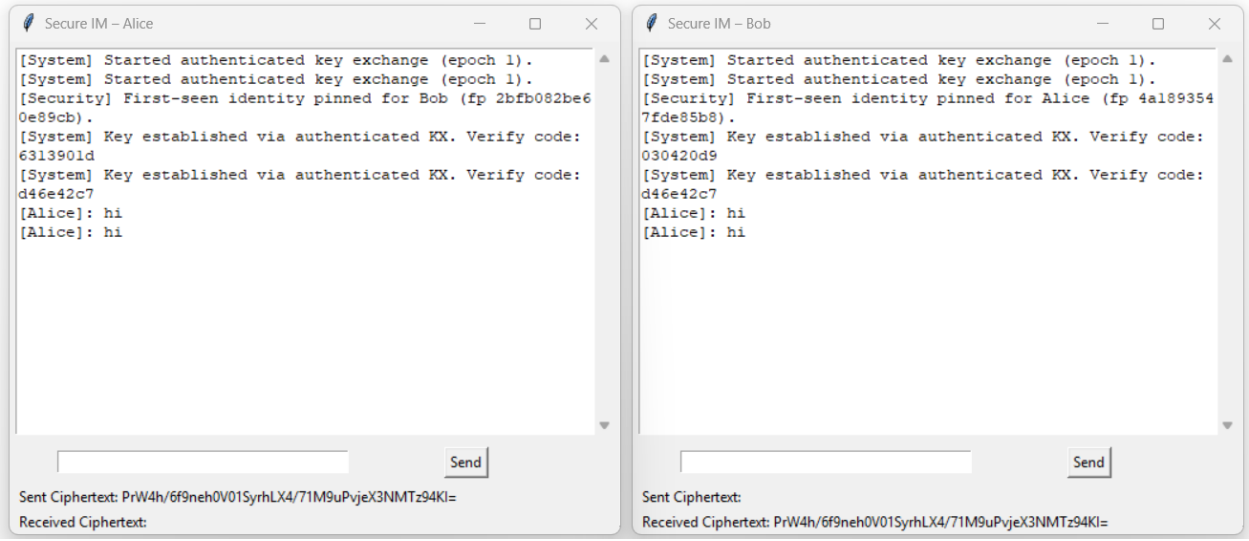

Figure 9: Issue 6 – Different ciphertext for repeated plaintext via random nonces.

Issue 7: Key management mechanism with periodic updates

A periodic rekey mechanism is implemented to limit key lifetime.

Major Functions:

- `msg_counter` in `server.py`: Maintains a counter incremented on each relayed message.
- `if msg_counter % KEY_UPDATE_INTERVAL == 0` in `server.py`: When threshold is reached, broadcasts `REKEY_NOW:{new_epoch}` to both clients.

- `if line.startswith(''REKEY_NOW'')` in `client.py`: Parses the new epoch and calls `_start_key_exchange(new_epoch)`.
- `_start_key_exchange(epoch)` and `_try_finalize_key_exchange(epoch)`: Perform fresh authenticated X25519 + HKDF for the new epoch, overwriting `self.k1`, `self.k2`, and `self.current_epoch`.

How It Achieves the Objective: The server tracks the number of messages relayed. After every 5 messages (`KEY_UPDATE_INTERVAL`), the server broadcasts `REKEY_NOW` with an incremented epoch number. Both clients receive this signal, initiate a fresh authenticated key exchange for that epoch, and derive new session keys via HKDF. The old session key is discarded. Subsequent messages use the new key.

Justification: Periodic key updates provide compartmentalization: compromise of a current key does not expose all past or future traffic. The amount of ciphertext protected under one key is bounded by the message threshold, limiting damage in a key compromise scenario.

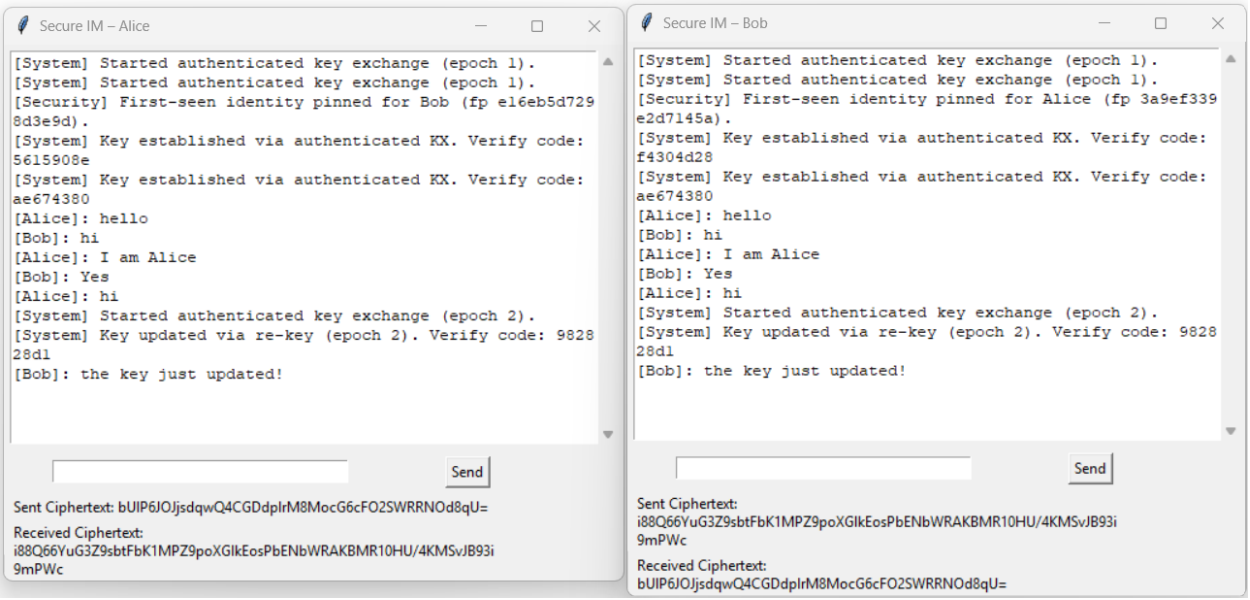

Figure 10: Issue 7 – Periodic key update mechanism with epoch rekeying.

4 Bonus 1: Designing a new Algorithm

Objective: If you can hide the detailed procedure of your encryption algorithm, how would you improve the security by designing a new algorithm?

The Procedure:
In new client protocol `client_custom_enc.py`...

- Use the HKDF session key to derive two distinct sub-keys: K_A and K_B.

- Encrypt the plaintext P using AES-256-CTR using K_A and a nonce N.
- Generate a Bit-Mask M by taking the first n bits of a SHA-256 hash of the current message counter with the session key, which then computes the cipher text.

Justification of Security:

- By using two different structures, we can mitigate the risk of a class break from AES. Furthermore, because the final output is masked by M from the HKDF state, the resulting ciphertext will pass all standard tests for randomness.
- In a standard XOR of two ciphers, if an attacker discovers one of them, they get the other for free. The addition of our Bit-Mask prevents this because M will change with every message counter refresh.

Justification of Efficiency:

- Because we are already performing an exchange with X25519, we do not need new keys. We can simply run the HKDF expand step twice to get our two keys, which adds minimal computational cost, providing significantly more security for the cost.
- Because AES-CTR functions as a stream cipher, the ciphertext is exactly the same length as the plaintext. For the GUI, this keeps the data clean and avoids bloat from block-cipher padding.

Major Functions:

- `custom_encrypt(plaintext)`: Derives `k1, k2` from HKDF, encrypts plaintext with AES-CTR using random nonce, then encrypts the CTR output with AES-CBC using random IV and PKCS7 padding.
- `custom_decrypt(ciphertext)`: Extracts nonce and IV from ciphertext header, reverses CBC with unpadding, then reverses CTR to recover plaintext.

How It Achieves the Objective: The dual-layer construction hides the cipher mode order. An attacker observing ciphertext cannot trivially determine whether CTR-inside-CBC or CBC-inside-CTR was used. Each layer contributes independent randomness (CTR nonce and CBC IV), and the composition provides defense-in-depth: breaking one layer does not immediately yield plaintext.

5 Bonus 2: Key Agreement Without Pre-shared Secret

Bonus 2 removes the assumption that Alice and Bob already share a password. Instead, they authenticate and negotiate a shared secret over an insecure channel.

Methodology

Objective: Establish a session key without pre-shared secret while still providing authentication, confidentiality, and periodic rekeying.

Protocol Design:

1. Each user has a long-term Ed25519 identity key pair stored locally.
2. For each key-exchange epoch, each side generates a fresh ephemeral X25519 key pair and random nonce.
3. Each side sends a signed key-exchange frame containing epoch, username, identity public key, ephemeral public key, and nonce.
4. Receiver verifies the signature using the included identity public key and enforces TOFU pinning on first contact.
5. Both sides compute the X25519 shared secret and derive session keys with HKDF-SHA256 using canonically ordered nonces as salt.
6. Verify code (truncated hash) is displayed so users can compare out-of-band if needed.

Implementation Details:

- Identity persistence: Ed25519 private key is stored as {`username`}`_ed25519.pem`.
- Trust model: first-seen peer identity is pinned in `trusted_identities.json`; subsequent mismatches are rejected.
- Rekey integration: server-triggered `REKEY_NOW` causes a fresh authenticated ephemeral exchange with incremented epoch.
- Key derivation: HKDF output is split into required symmetric keys for the active client mode.

Security Rationale:

- Authentication: Ed25519 signatures bind exchanged ephemeral keys to persistent identities.
- Forward secrecy: ephemeral X25519 keys are regenerated per epoch and not reused.
- MitM resistance after first trust: identity pinning detects key substitution attempts on later sessions.
- Key freshness: periodic epoch rekeying limits the amount of data protected under one key.

Limitations: TOFU does not protect against a man-in-the-middle attack during the first ever contact. This is mitigated operationally by manual verify-code comparison or pre-distribution of identity fingerprints.

Runtime Example

```
# Start server
python3 server.py --port 8000

# Client 1
python3 client.py Alice --host 127.0.0.1 --port 8000 --initiate

# Client 2
python3 client.py Bob --host 127.0.0.1 --port 8000
```

Expected results:

- key-exchange frames are exchanged and signatures are accepted,

- verify code is printed on both clients,
- encrypted chat works normally,
- periodic rekey output appears after threshold messages.

Major Functions:

- `_load_or_create_identity_key()`: Loads or generates Ed25519 long-term identity key, persisted as {`username`}`_ed25519.pem`.
- `_start_key_exchange(epoch)`: Generates ephemeral X25519 pair, creates signed KX_INIT frame, sends to peer.
- `_try_finalize_key_exchange(epoch)`: Verifies peer signature, performs X25519 ECDH, derives keys via HKDF with canonical salt ordering, and pins identity via TOFU.
- `_pin_or_verify_identity()`: Stores first peer identity in `trusted_identities.json`; subsequent mismatches trigger security error.

How It Achieves the Objective: Without a pre-shared password, each peer has only a long-term Ed25519 identity key. The protocol uses this to sign ephemeral X25519 keys, binding identity to the session. Both peers compute the same shared secret via X25519 and expand it into session keys via HKDF. TOFU pinning stores the first peer identity; later sessions reject any identity change, protecting against mid-session MitM attacks.

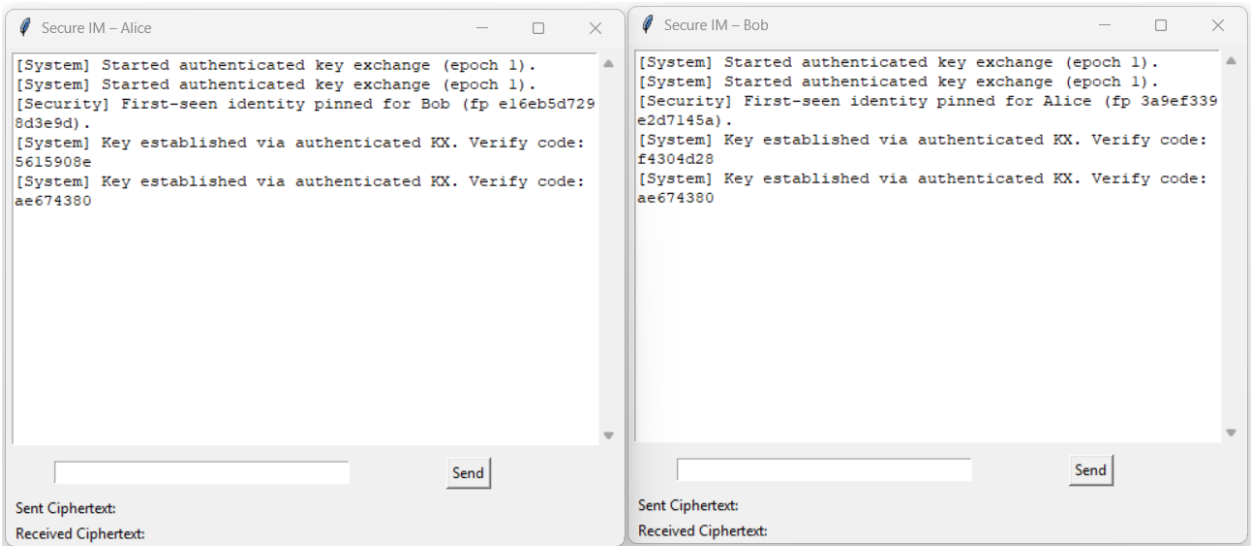

Figure 11: Bonus 2 – Authenticated key exchange and matching session keys.


```
{
  "Alice": "pmlxFbTsO2FRbSgZ4opmt8nlygFgwkiZZaUW22gEQvY=",
  "Bob": "Buv9ySKI8G8YOZzbj0hmGEKlBwA3sSwnd1b2TVGLklg="
}
```


Figure 12: Bonus 2 – Pinned keys.

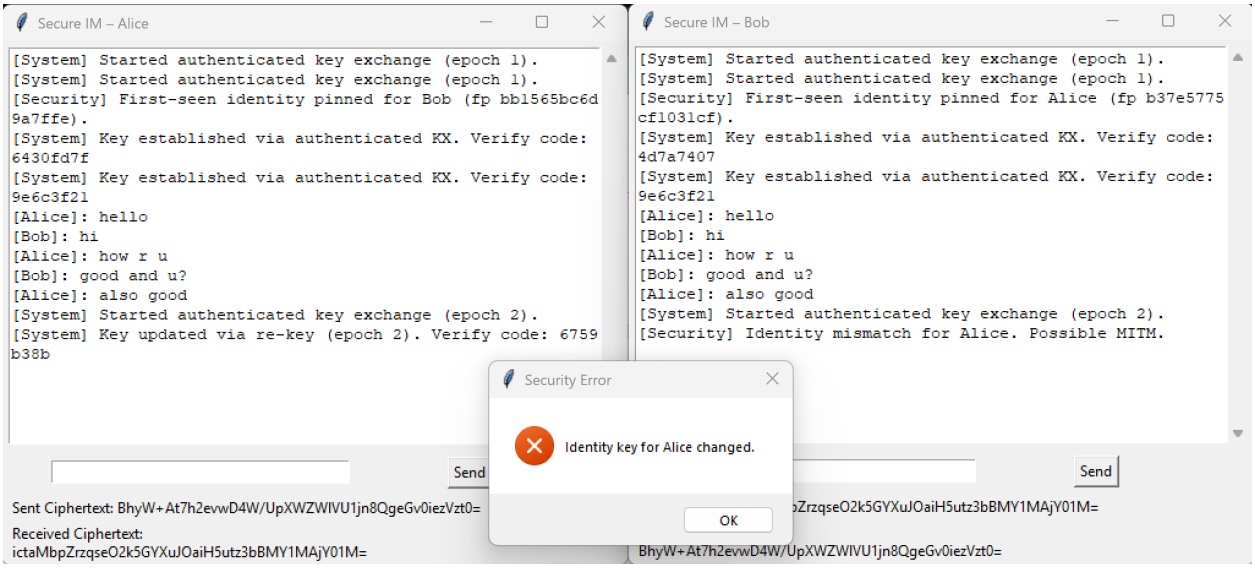

Figure 13: Bonus 2 – TOFU identity trust workflow and mismatch detection upon Alice key change.

6 Security Analysis and Limitations

This section summarizes security properties and known limitations.

Security Properties

- Confidentiality: Provided by AES-GCM encryption using per-session keys.
- Integrity and authenticity: AES-GCM tags and optional signature of ephemeral keys.
- Forward secrecy: Provided when ephemeral ECDH keys are used.
- Replay protection: Sequence numbers and simple stateful checks.

Limitations and TOFU Discussion

Default configuration uses TOFU (Trust On First Use). TOFU means:

- On first contact, the remote public key is accepted and stored locally.
- If an attacker performs a MitM at first contact, the attacker can impersonate the remote party.

Mitigations and notes:

- For stronger authentication, enable signed ephemeral keys using pre-shared long-term identity keys or use a PKI.
- Display and compare session key fingerprints (SHA-256 truncated) to allow manual out-of-band verification.

Code Evidence:

- `ed25519.Ed25519PublicKey.from_public_bytes(peer_pub).verify(sig, payload)`: Verifies ephemeral key signature; raises exception on mismatch, triggering security error.

- `_pin_or_verify_identity()`: Compares stored trusted key with incoming key; if mismatch, displays TOFU violation and closes connection.

```
if trusted and trusted != peer_b64:
    self.display_message(f"[Security] Identity mismatch for {peer_username}. Possible MITM.")
    self.running = False
    self.sock.close()
    self.master.after(0, lambda: messagebox.showerror("Security Error", f"Identity key for {peer_username} changed."))
    return False
```

Figure 14: Security limitation and mitigation code.

7 Conclusion

This project successfully demonstrates a complete secure messaging system that addresses all seven core design issues while implementing both bonus objectives, providing AES-256 encryption, HKDF key derivation from ephemeral X25519 shared secrets, authenticated encryption via AES-GCM, a transparent Tkinter GUI, reliable TCP relay transport, randomized ciphertexts, and periodic rekeying for forward secrecy. Bonus 1 introduces a custom dual-layer encryption scheme (CTR + CBC) for defense-in-depth, while Bonus 2 implements authenticated key establishment without pre-shared passwords using Ed25519 identities and TOFU identity pinning. The codebase includes two complete implementations (`SharedPassword/` and `Bonus2-KeyEnc/`) with modular design separating cryptographic primitives from network transport. Key limitations such as the TOFU vulnerability during first contact are documented with practical mitigations suitable for deployment. This implementation demonstrates practical application of modern cryptographic concepts including symmetric and asymmetric cryptography, key derivation, authenticated encryption, forward secrecy, and threat modeling, all of which are essential components of secure communication systems.